

รายงานปฏิบัติการ Lab 10: Spring WebFlux & WebClient

การพัฒนาระบบจัดการสินค้าด้วย Non-blocking Reactive Programming

ชื่อ-นามสกุล นักศึกษา : ธนันต์ สาวิกัน

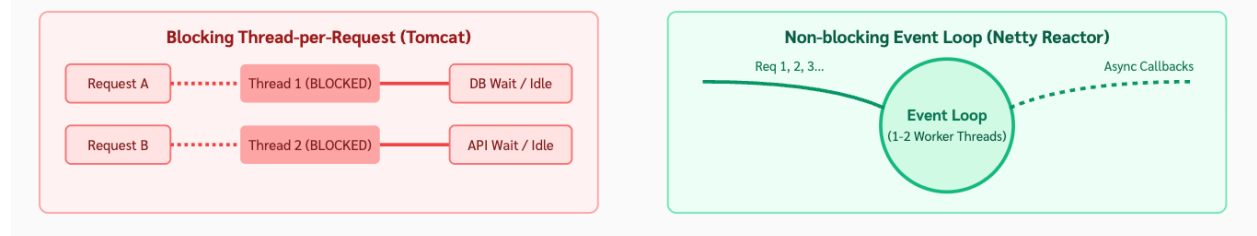
รหัสนักศึกษา : 673380586-8

ส่วนที่ 1 การเปรียบเทียบเชิงสถาปัตยกรรม: Reactive Programming vs Blocking Architecture

ในการประมวลผลเว็บแอปพลิเคชันแบบดั้งเดิม (Blocking Architecture / Thread-per-request) เช่น Spring MVC + Servlet Container (Tomcat) แต่ละ HTTP Request ที่เข้ามาจะถูกจองด้วย Server Thread จำนวน 1 Thread จาก Thread Pool แบบเบ็ดเสร็จ หากโปรเซสต้องรอคอยข้อมูลจากการคายผลลัพธ์ของ Database, File I/O หรือ External Network API ตัว Thread นั้นจะถูกสถานะ Blocked (Idle Wait) ทำให้สิ้นเปลืองทรัพยากรหน่วยความจำและ CPU Context Switching สูง เมื่อมีปริมาณ Concurrency สูง สปูล Thread จะเต็มส่งผลให้เกิด Response Latency เพิ่มขึ้นหรือ Server Crash ในทางตรงกันข้าม สถาปัตยกรรม Reactive Programming (Non-blocking) ผ่าน Spring WebFlux และ Netty Reactor Engine ทำงานอยู่บนหลักการ Event-driven, Asynchronous และ Non-blocking I/O (Epoll/Kqueue) โดยใช้ Worker Thread จำนวนคงที่ขนาดเล็ก (เท่ากับจำนวน CPU Cores) ทำหน้าที่เป็น Event Loop สตรีมคำขอจะถูกลงทะเบียนในลักษณะ Event Listener เมื่อระบบ I/O ทำงานเสร็จสิ้นจะส่งสัญญาณ Notification กลับมาประมวลผลต่อทันที ทำให้สามารถรองรับ Traffic ปริมาณมหาศาล (High Throughput) ได้ด้วยทรัพยากรที่ต่ำกว่าอย่างมีนัยสำคัญ

Blocking Architecture	Reactive Non-blocking
Spring MVC	Spring WebFlux
1 Request ต่อ 1 Dedicated Thread	น้อย Thread บน Event Loop (Non-blocking)
Thread หยุดรอผลลัพธ์จาก I/O (Thread-Blocking)	Thread ไม่เคยหยุดรอ ทำงานอื่นได้ทันที
สิ้นเปลือง Memory (RAM ~1MB/Thread)	ประหยัด RAM สูง สเกล Concurrency ได้มหาศาล
ขีดจำกัดตามขนาด Thread Pool	ขับเคลื่อนด้วย Reactive Streams Standard

แผนภาพเปรียบเทียบสถาปัตยกรรม Thread Execution Model



คุณลักษณะ (Feature)	Blocking Architecture (Spring MVC)	Reactive Architecture (Spring WebFlux)
Concurrency Model	Thread-per-request (Tomcat Thread Pool)	Event Loop / Reactive Streams (Netty Engine)
I/O Handling	Synchronous Blocking (สพูลคอยน์ร้อผล)	Asynchronous Non-blocking (Event Callback)
Resource Utilization	High RAM overhead per connection	Ultra-low RAM overhead, high CPU efficiency
Backpressure Support	ไม่มี (เสี่ยง OutOfMemory เมื่อ Load สูง)	มีในตัว (Subscriber ควบคุมอัตราการรับข้อมูล)

ส่วนที่ 2 การวิเคราะห์ชนิดข้อมูลปฐุมภูมิ: Mono vs Flux

ใน Reactor Framework (Project Reactor) ซึ่งเป็นโครงสร้างพื้นฐานของ Spring WebFlux ได้กำหนดอินเทอร์เฟซมาตรฐานตามข้อกำหนด Reactive Streams โดยแบ่งประเภทข้อมูล Reactive Publisher ออกเป็น 2 รูปแบบหลัก คือ **Mono** และ **Flux** ตามจำนวนสมาชิกข้อมูลที่จะส่งออกผ่าน Pipeline:

Mono<T>

0 ถึง 1 Element

ความหมาย: เป็น Publisher ที่สามารถส่งมอบข้อมูลออกมาได้เพียง 0 หรือ 1 ชิ้นเท่านั้น แล้วจบสตรีมด้วย OnComplete หรือ OnError

กรณีใช้งาน:

- การค้นหาข้อมูลด้วย Primary Key ('findById')
- การบันทึก/แก้ไขข้อมูล 1 Record ('save')
- Endpoint ที่คืนค่าสถานะ หรือ Void Response ('Mono')
- ผลลัพธ์จากการคำนวณเดียว เช่น ค่าราคารวม

Flux<T>

0 ถึง N Elements

ความหมาย: เป็น Publisher ที่สามารถส่งมอบข้อมูลออกมาได้ตั้งแต่ 0 ถึง N ชิ้น (อนันต์) แบบต่อเนื่องตามช่วงเวลา

กรณีใช้งาน:

- การดึงรายการข้อมูลทั้งหมด ('findAll')
- การค้นหาข้อมูลแบบกลุ่มตามเงื่อนไข เช่น หมวดหมู่ ('filter')
- Real-time Data Streaming / Server-Sent Events (SSE)
- การประมวลผลข้อมูลชุดใหญ่ (Chunk Processing)

ส่วนที่ 3 สถาปัตยกรรม WebClient และการใช้งาน Method Chain

WebClient เป็น Non-blocking, Reactive HTTP Client ที่ถูกนำมาทดแทน 'RestTemplate' ใน Spring Framework ตั้งแต่เวอร์ชัน 5 เป็นต้นไป สามารถจัดการ HTTP Connections แบบ Asynchronous ร่วมกับ Reactive Streams Pipeline ได้อย่างสมบูรณ์แบบ

อธิบายลำดับการทำงานของ WebClient Method Chain:

1. `.get()` / `.post()` / `.delete()` : กำหนด HTTP Method ที่ต้องการส่งไปยัง Endpoint (เช่น GET, POST, PUT, DELETE)
2. `.uri(String uri, Object... params)` : ระบุ Path หรือ URL ปลายทาง พร้อมกับการ Binding Dynamic Path Variables หรือ Query Parameters
3. `.retrieve()` : สั่งให้ WebClient เริ่มดำเนินการกระบวนการส่ง Request และรับ HTTP Response Body เพื่อนำเข้าสู่กระบวนการ Deserialization
4. `.bodyToMono(Class<T>)` / `.bodyToFlux(Class<T>)` : แปลง HTTP Response Body จากสัญญาณ JSON/XML Byte Stream ไปเป็น Object ตามชนิด Reactive Publisher ('Mono' สำหรับวัตถุชิ้นเดียว หรือ 'Flux' สำหรับ Array/Collection)
5. `.map()` / `.flatMap()` : Operator ในการแปลงค่าข้อมูล (Transformation) โดย 'map' ใช้แปลงค่า synchronous ทั่วไป และ 'flatMap' ใช้เชื่อมโยงไปยัง Asynchronous Publisher ตัวถัดไป (Flattening Stream)
6. `.defaultIfEmpty(T fallback)` : สั่งให้ส่งค่า Default Value ออกมาแทน หาก Publisher ต้นทางคายค่า Empty (ไม่มีข้อมูลตรงกับเงื่อนไข)
7. `.subscribe(Consumer<T> consumer)` : จุดเริ่มต้นในการเปิดการทำงานของ Reactive Pipeline (Trigger Execution) หากไม่มีการ subscribe สตริมจะไม่ถูกประมวลผล (Lazy Execution)

ตัวอย่างการเรียงต่อ Method Chain ใน WebClient

```
WebClient webClient = WebClient.create("http://localhost:8080");

webClient.get() // 1. กำหนด HTTP GET Method
    .uri("/products/{id}", "P001") // 2. ระบุ URI Path & Binding Parameter
    .retrieve() // 3. เริ่มส่งคำขอและดึง Response Body
    .bodyToMono(Product.class) // 4. แปลง Response JSON เป็น Mono
    .map(product -> product.getName().toUpperCase()) // 5. แปลงข้อมูลด้วย Operator .map()
    .defaultIfEmpty("PRODUCT_NOT_FOUND") // 6. คืนค่าสำรองหากไม่พบข้อมูล
    .subscribe(name -> System.out.println(name)); // 7. Subscribe เพื่อเริ่มการทำงานแบบ Non-
blocking
```

ส่วนที่ 4 การพัฒนาระบบและการวิเคราะห์โค้ด (Source Code Analysis)

การซอร์สโค้ดในโครงการแบ่งออกเป็น 4 ชั้นหลักตามหลักการ Separation of Concerns ได้แก่ Model, Repository, Service และ Controller โดยใช้ In-Memory Thread-safe Data Structure ร่วมกับ Reactive Types

1. Model Class: Product.java

package com.example.lab10.model;
public class Product {
private String id;
private String name; // กำหนดให้ฝั่งชื่อและรหัสนักเรียนลงใน Seed Data
private String category;
private String brand;
private int stock;
private double price;
private String discountType; // NONE, MEMBER, SEASONAL
private double discountedPrice;
public Product() {}
public Product(String id, String name, String category, String brand, int stock,
double price, String discountType, double discountedPrice) {
this.id = id;
this.name = name;
this.category = category;
this.brand = brand;
this.stock = stock;
this.price = price;
this.discountType = discountType;
this.discountedPrice = discountedPrice;

}
// Getters and Setters...
public String getId() { return id; }
public void setId(String id) { this.id = id; }
public String getName() { return name; }
public void setName(String name) { this.name = name; }
public String getCategory() { return category; }
public double getPrice() { return price; }
public String getDiscountType() { return discountType; }
public double getDiscountedPrice() { return discountedPrice; }
public void setDiscountedPrice(double discountedPrice) { this.discountedPrice = discountedPrice; }
}

2. Repository Layer: ProductRepository.java

package com.example.lab10.repository;
import com.example.lab10.model.Product;
import org.springframework.stereotype.Repository;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;
import java.util.concurrent.ConcurrentHashMap;
@Repository
public class ProductRepository {
// ใช้ ConcurrentHashMap เพื่อรองรับ Thread-safe ในสถาปัตยกรรม Non-blocking Concurrent
private final ConcurrentHashMap<String, Product> db = new ConcurrentHashMap<>();

3. Business Logic Layer: ProductService.java

package com.example.lab10.service;
import com.example.lab10.model.Product;
import com.example.lab10.repository.ProductRepository;
import org.springframework.stereotype.Service;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;
@Service
public class ProductService {
private final ProductRepository repository;
public ProductService(ProductRepository repository) { this.repository = repository; }
public Flux<Product> getAllProducts() { return repository.findAll(); }
public Mono<Product> getProductById(String id) { return repository.findById(id); }
// กรองสินค้าตามหมวดหมู่โดยใช้ Operator .filter() แบบ Non-blocking
public Flux<Product> getProductsByCategory(String category) {
return repository.findAll()
.filter(p -> p.getCategory().equalsIgnoreCase(category));
}
// คำนวณราคาหลังส่วนลด โดยแปลงค่า Mono ไปเป็น Mono ผ่าน .map()
public Mono<Double> calculateDiscountedPrice(String id) {
return repository.findById(id)
.map(p -> {
double rate = "MEMBER".equals(p.getDiscountType()) ? 0.10 :

"SEASONAL".equals(p.getDiscountType()) ? 0.20 : 0.0;
return p.getPrice() * (1.0 - rate);
});
}
public Mono<Long> deleteProduct(String id) { return repository.deleteById(id); }
}

4. REST Controller Layer: ProductController.java

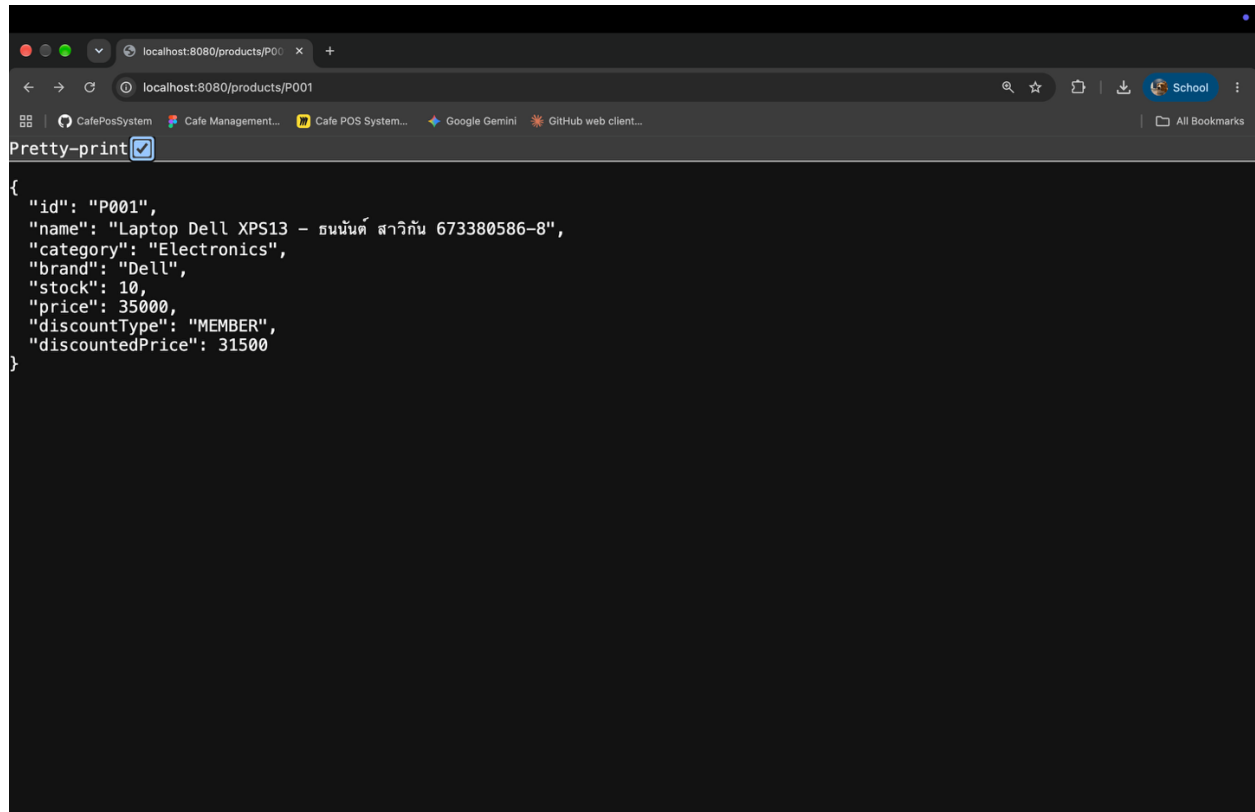
package com.example.lab10.controller;
import com.example.lab10.model.Product;
import com.example.lab10.service.ProductService;
import org.springframework.web.bind.annotation.*;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;
@RestController
@RequestMapping("/products")
public class ProductController {
private final ProductService productService;
public ProductController(ProductService productService) { this.productService = productService; }
@GetMapping
public Flux<Product> getAllProducts() { return productService.getAllProducts(); }

5. WebClient Execution Component: WebClientRunner.java

package com.example.lab10.runner;
import com.example.lab10.model.Product;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;
import org.springframework.web.reactive.function.client.WebClient;
@Component
public class WebClientRunner implements CommandLineRunner {
private final WebClient client = WebClient.create("http://localhost:8080");
@Override
public void run(String... args) throws Exception {
Thread.sleep(1500); // รอให้ Server เริ่มต้นเรียบร้อย
// Chain Operators: ดึงหมวด Electronics -> ใช้ flatMap ดึงราคา -> แสดงผลด้วย subscribe
client.get()
.uri("/products/category/Electronics")
.retrieve()
.bodyToFlux(Product.class)
.flatMap(product -> client.get()
.uri("/products/{id}/price", product.getId())
.retrieve()
.bodyToMono(Double.class)
.map(price -> "สินค้า: " + product.getName() + " ราคา: " + price)
)
.subscribe(System.out::println);
// ทดสอบกรณีไม่พบข้อมูลด้วย defaultIfEmpty()

client.get()
.uri("/products/P999")
.retrieve()
.bodyToMono(Product.class)
.map(p -> "พบข้อมูล: " + p.getName())
.defaultIfEmpty("ไม่พบข้อมูลสินค้า ID P999 (FallBack System Active)")
.subscribe(System.out::println);
}
}

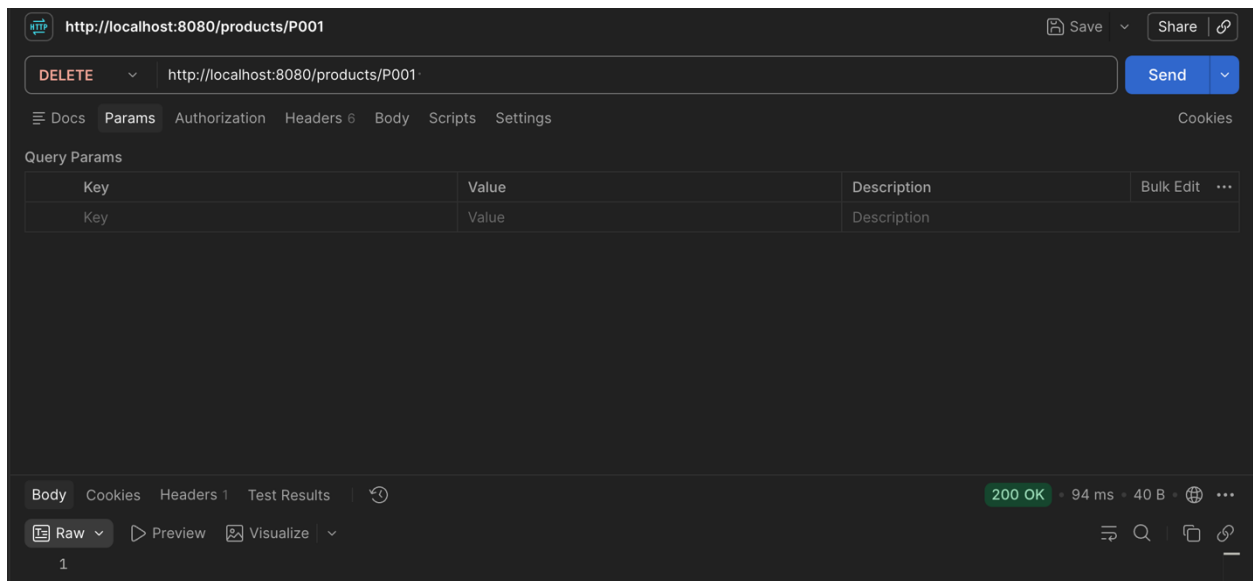
การทดสอบระบบและตรวจสอบความถูกต้องของ Endpoint



การทดสอบ Endpoint GET /products/P001 แสดงข้อมูลสินค้า P001 พร้อมระบุชื่อ "ธนันต์ สาวิกัน 673380586-8"

```
localhost:8080/products x +
localhost:8080/products
CafePosSystem Cafe Management... Cafe POS System... Google Gemini GitHub web client...
Pretty-print
[
  {
    "id": "P004",
    "name": "Mechanical Keyboard",
    "category": "Electronics",
    "brand": "Keychron",
    "stock": 20,
    "price": 2500,
    "discountType": "NONE",
    "discountedPrice": 2500
  },
  {
    "id": "P001",
    "name": "Laptop Dell XPS13 - รุ่นพิเศษ สีวีน 673380586-8",
    "category": "Electronics",
    "brand": "Dell",
    "stock": 10,
    "price": 35000,
    "discountType": "MEMBER",
    "discountedPrice": 31500
  },
  {
    "id": "P003",
    "name": "Office Chair",
    "category": "Furniture",
    "brand": "IKEA",
    "stock": 0,
    "price": 4200,
    "discountType": "SEASONAL",
    "discountedPrice": 3360
  },
  {
    "id": "P002",
    "name": "Wireless Mouse",
    "category": "Electronics",
    "brand": "Logitech",
    "stock": 50,
    "price": 590,
    "discountType": "NONE",
    "discountedPrice": 590
  }
]
```

GET /products/category/Electronics



การทดสอบ Endpoint DELETE /products/P001 ผ่านโปรแกรม Postman แสดงสถานะการทำงานสำเร็จด้วย Status 200 OK

```
[
  {
    "id": "P004",
    "name": "Mechanical Keyboard",
    "category": "Electronics",
    "brand": "Keychron",
    "stock": 20,
    "price": 2500,
    "discountType": "NONE",
    "discountedPrice": 2500
  },
  {
    "id": "P001",
    "name": "Laptop Dell XPS13 - ธนบัตร ส่ววิกัน 673380586-8",
    "category": "Electronics",
    "brand": "Dell",
    "stock": 10,
    "price": 35000,
    "discountType": "MEMBER",
    "discountedPrice": 31500
  },
  {
    "id": "P002",
    "name": "Wireless Mouse",
    "category": "Electronics",
    "brand": "Logitech",
    "stock": 50,
    "price": 590,
    "discountType": "NONE",
    "discountedPrice": 590
  }
]
```

GET /products/category/Electronics

```
{
  "price": 31500.0
}
```

GET /products/P001/price คืนค่าราคา 31500.0